Kiste Phase 3 Spec

Multi-Repo Workspace + Token Access Graph

Phase 3 upgrades Kiste from scanning one repo at a time to understanding a full product made from many repos.

Phase	Focus
Phase 1	Local Docker repo scan + token inspection
Phase 2	GitHub/Hugging Face remote repo scan + CI testing
Phase 3	Multi-repo workspace + system graph + token access matrix

1. Goal

Kiste Phase 3 connects multiple GitHub and Hugging Face repos into one workspace, scans them together, maps their dependencies, checks token access, and reports whether the whole system is ready for Kubernetes generation.

Phase 3 does not deploy yet. It prepares the full multi-repo system for later Kubernetes, CI/CD, security, cost, and GPU-aware generation.

2. Main Problem

Real systems are often split across many repos. Kiste needs to understand that these are not separate projects, but one connected system.

Repo type	Purpose	Example
Frontend	User interface	React, Next.js, Vue
Backend API	Main server application	FastAPI, Django, Express, Go API
Worker	Background jobs	Celery, queue consumer, batch inference
ML model	Model artifacts or model code	Hugging Face model repo
Dataset	Training or evaluation data	HF dataset, CSV, Parquet, images
Infra	Infrastructure as code	Terraform, Helm, Kubernetes YAML
Shared library	Reusable code	Python package, npm package, Go module
Spec/docs	Architecture and contracts	OpenAPI, ADRs, security policy

3. Workspace Concept

A workspace represents one product or system. It contains many repos, their providers, their roles, token assignments, and repo-to-repo connections.

```
workspace:
  name: robot-platform
  environment: dev

defaults:
  tokens:
    github: github-read-dev
    hf: hf-read-dev

repos:
  - name: frontend
    provider: github
    repo: khoa/robot-frontend
    type: frontend

  - name: api
    provider: github
    repo: khoa/robot-api
    type: backend
    token: github-build-dev

  - name: worker
    provider: github
    repo: khoa/robot-worker
    type: worker
    token: github-build-dev

  - name: model
    provider: hf
    repo: khoa/robot-vision-model
    type: model

  - name: infra
    provider: github
```

```
    repo: khoa/robot-infra
    type: infrastructure
    token: github-infra-dev
```

4. Phase 3 Command Tree

```
kiste
  workspace
    init
    add
    list
    remove
    scan
    graph
    check
    summary
    token
      assign
      check
      matrix
  connect
  disconnect
```

5. Workspace Commands

```
# Create workspace
kiste workspace init --name robot-platform

# Add repos
kiste workspace add github khoa/robot-api --name api --type backend
kiste workspace add github khoa/robot-frontend --name frontend --type frontend
kiste workspace add github khoa/robot-worker --name worker --type worker
kiste workspace add hf khoa/robot-vision-model --name model --type model
kiste workspace add github khoa/robot-infra --name infra --type infrastructure

# Inspect workspace
kiste workspace list
kiste workspace scan
kiste workspace graph
kiste workspace check --target k8s
```

6. Manual Connection Commands

Kiste should infer connections automatically, but the user must be able to define or override relationships manually.

```
kiste connect frontend api --relation calls
kiste connect api postgres --relation uses-db
kiste connect api redis --relation uses-cache
kiste connect worker queue --relation consumes
kiste connect api model --relation loads-model
kiste disconnect api model
```

7. Auto-Detected Connections

Signal found by Kiste	Inferred connection
NEXT_PUBLIC_API_URL / VITE_API_URL / REACT_APP_API_URL	frontend -> api
DATABASE_URL	api -> database
REDIS_URL	api/worker -> redis
CELERY_BROKER_URL	worker -> queue
HF_MODEL_ID	api/worker -> Hugging Face model
docker-compose.yml service links	service graph
package.json dependency	repo -> shared npm library
requirements.txt dependency	repo -> shared Python package

Signal found by Kiste	Inferred connection
Terraform/Helm/K8s files	infra repo -> cluster/cloud resources

8. Workspace Output Files

```
.kiste/
  workspace.json
  graph.json
  token-matrix.json
  workspace-report.json
  repos/
    frontend.spec.json
    api.spec.json
    worker.spec.json
    model.spec.json
    infra.spec.json
```

9. Example Workspace JSON

```
{
  "workspace": {
    "name": "robot-platform",
    "environment": "dev"
  },
  "repos": [
    {
      "name": "frontend",
      "provider": "github",
      "repo": "khoa/robot-frontend",
      "type": "frontend",
      "token": "github-read-dev"
    },
    {
      "name": "api",
      "provider": "github",
      "repo": "khoa/robot-api",
      "type": "backend",
      "token": "github-build-dev"
    },
    {
      "name": "model",
      "provider": "hf",
      "repo": "khoa/robot-vision-model",
      "type": "model",
      "token": "hf-read-dev"
    }
  ],
  "connections": [
    {"from": "frontend", "to": "api", "relation": "calls"},
    {"from": "api", "to": "postgres", "relation": "uses-db"},
    {"from": "api", "to": "redis", "relation": "uses-cache"},
    {"from": "api", "to": "model", "relation": "loads-model"}
  ]
}
```

10. Multi-Repo Token System

Tokens in Phase 3 are workspace-level capabilities. Kiste should not only ask whether a GitHub token exists. It should check whether this exact token can perform this exact action on this exact repo before expiry.

```
tokens:
  - name: github-read-dev
    provider: github
    scopes:
      - contents:read
    expires_at: 2026-08-01
    secret_reference: env://KISTE_TOKEN_GITHUB_READ_DEV

  - name: github-build-dev
    provider: github
```

```
    scopes:
      - contents:read
      - packages:write
      - actions:read
    expires_at: 2026-08-01
    secret_reference: env://KISTE_TOKEN_GITHUB_BUILD_DEV

  - name: hf-read-dev
    provider: hf
    scopes:
      - repo:read
    expires_at: 2026-08-01
    secret_reference: env://KISTE_TOKEN_HF_READ_DEV
```

Raw tokens are not stored in workspace files. They should come from environment variables.

```
export KISTE_TOKEN_GITHUB_READ_DEV="..."
export KISTE_TOKEN_GITHUB_BUILD_DEV="..."
export KISTE_TOKEN_HF_READ_DEV="..."
```

11. Token Assignment Commands

```
kiste workspace token assign frontend github-read-dev
kiste workspace token assign api github-build-dev
kiste workspace token assign worker github-build-dev
kiste workspace token assign model hf-read-dev

kiste workspace token check
kiste workspace token check --repo api
kiste workspace token check --target scan
kiste workspace token check --target build
kiste workspace token matrix
```

12. Token Access Matrix

Repo	Provider	Token	Scan	Build	Package	Expires
frontend	github	github-read-dev	OK	NO	NO	2026-08-01
api	github	github-build-dev	OK	OK	OK	2026-08-01
worker	github	github-build-dev	OK	OK	OK	2026-08-01
model	hf	hf-read-dev	OK	N/A	N/A	2026-08-01
infra	github	missing	NO	NO	NO	-

13. Internal Permission Model

Target	Meaning
scan	Read repo metadata, important files, Dockerfile, package files, README
build	Read source, read workflow files, trigger CI/CD job
package	Push container image or package artifact
secret	Read/create repo secrets; prepared but not fully implemented yet
deploy	Access infra repo, Kubernetes cluster, cloud account; prepared for later phases

Phase 3 should mainly support scan, build, and package. Secret and deploy can exist in the model but do not need full implementation yet.

14. Workspace Graph

```
frontend -> api
api -> postgres
api -> redis
api -> model
```

```
worker -> redis
worker -> model
infra -> cluster

[frontend repo]
      |
      v
[api repo] ---> [postgres]
      |
      v
[HF model repo]

[worker repo] ---> [redis]
```

This graph becomes the base for Kubernetes manifests, NetworkPolicy, Ingress rules, CI/CD build order, secret mapping, cost estimation, GPU planning, and security review.

15. Workspace Scan Pipeline

```
workspace.json
   |
   v
repo list
   |
   v
provider selection
   |
   v
GitHub/HF scan
   |
   v
individual repo specs
   |
   v
connection inference
   |
   v
system graph
   |
   v
workspace report
```

16. Workspace Readiness Check

```
kiste workspace check --target k8s

Workspace: robot-platform

Repos:
OK frontend scanned
OK api scanned
OK worker scanned
OK model scanned
NO infra repo access denied

System graph:
OK frontend -> api
OK api -> postgres
OK api -> redis
OK api -> model
WARN worker has no queue configured

Token access:
OK frontend token can scan
OK api token can scan/build/package
OK worker token can scan/build/package
OK model token can scan
NO infra token missing

Kubernetes readiness:
OK frontend can generate Deployment + Service + Ingress
OK api can generate Deployment + Service
OK worker can generate Deployment
WARN model is external HF dependency
NO ingress host missing
```

17. Architecture

```
Kiste CLI
   |
   v
Workspace Manager
   |
   v
Repo Registry
   |
   v
Provider Scanner
   |-- Local Provider
   |-- GitHub Provider
   |-- Hugging Face Provider
   |
   v
Spec Builder
   |
   v
Connection Inference Engine
   |
   v
Token Access Checker
   |
   v
System Graph Builder
   |
   v
Workspace Readiness Checker
   |
   v
Report Generator
```

18. Data Model

Repo object

```
{
  "name": "api",
  "provider": "github",
  "repo": "khoa/robot-api",
  "type": "backend",
  "token": "github-build-dev",
  "spec_path": ".kiste/repos/api.spec.json"
}
```

Connection object

```
{
  "from": "frontend",
  "to": "api",
  "relation": "calls",
  "source": "auto-detected",
  "confidence": 0.82
}
```

Token access object

```
{
  "repo": "api",
  "provider": "github",
  "token": "github-build-dev",
  "checks": {
    "scan": true,
    "build": true,
    "package": true,
    "secret": false,
    "deploy": false
  },
  "expires_at": "2026-08-01",
  "status": "active"
}
```

19. What Phase 3 Should Not Do Yet

- No full Kubernetes generation for all repos

- No real multi-repo deployment

- No automatic Terraform apply

- No production secrets manager

- No full cost optimizer

- No full GPU scheduler

- No Karmada multi-cluster scheduling

- No automatic repo modification unless explicitly requested

20. Acceptance Criteria

```
kiste workspace init --name robot-platform

kiste workspace add github khoa/robot-frontend --name frontend --type frontend
kiste workspace add github khoa/robot-api --name api --type backend
kiste workspace add github khoa/robot-worker --name worker --type worker
kiste workspace add hf khoa/robot-vision-model --name model --type model

kiste workspace token assign frontend github-read-dev
kiste workspace token assign api github-build-dev
kiste workspace token assign worker github-build-dev
kiste workspace token assign model hf-read-dev

kiste workspace scan
kiste workspace graph
kiste workspace token check
kiste workspace token matrix
kiste workspace check --target k8s
```

Phase 3 is complete when Kiste can produce workspace.json, graph.json, token-matrix.json, individual repo specs, a system dependency graph, repo-to-repo connection map, multi-repo token access report, and multi-repo Kubernetes readiness report.

21. One-Sentence Phase 3 Spec

Kiste Phase 3 connects multiple GitHub and Hugging Face repos into one workspace, scans them together, builds a system dependency graph, checks token access across all repos, and reports whether the whole multi-repo application is ready for Kubernetes generation.